TypeScript fejlesztés Visual Studio Code környezetben

Hallgatói útmutató Windows rendszerhez

Az útmutató bemutatja a fejlesztői környezet telepítését, egy egyszerű HTML és TypeScript projekt létrehozását, valamint a TypeScript JavaScriptre fordítását. A lépések végére egy böngészőben működő alkalmazást készítünk, majd beállítjuk az automatikus fordítást és az oldalfrissítést.

1 Visual Studio Code telepítése

Nyissuk meg a https://code.visualstudio.com/ weboldalt, töltsük le a Windows-telepítőt, majd indítsuk el. Ha a telepítő felajánlja, jelöljük be az Add to PATH opciót. A fájlok és mappák helyi menüből történő megnyitását is engedélyezhetjük.

A VS Code beépített TypeScript-szerkesztési támogatást tartalmaz, de a JavaScript előállításához külön TypeScript-fordítót kell telepítenünk.

2 Node.js és npm telepítése

A https://nodejs.org/ weboldalról töltsük le az LTS jelölésű Node.js-verziót. Telepítsük az alapértelmezett beállításokkal az npm csomagkezelő is települ.

Zárjuk be, majd indítsuk újra a VS Code-ot. A Terminal → New Terminal menüponttal nyissunk terminált és külön-külön futtassuk:

- node -v
- npm -v

Ha mindkét parancs verziószámot ír ki, a telepítés sikeres. A pontos verziószám a telepített kiadástól függ.

3 A TypeScript fordító telepítése

- npm install -g typescript
- tsc -v

Az első parancs telepít, a második ellenőriz. A -g globális telepítést jelent: a tsc fordító több projektmappából is elérhető. Nagyobb projekteknél a projektenkénti telepítés is gyakori: npm install --save-dev typescript. Ebben az útmutatóban a globális telepítést használjuk.

4 Az első projekt létrehozása

Hozzunk létre egy TypeScript_Gyakorlat01 nevű mappát. A VS Code-ban válasszuk a File → Open Folder menüpontot és nyissuk meg ezt a mappát. Az Explorer panel New File gombjával hozzuk létre az index.html és main.ts fájlokat.

Az index.html tartalma

```
<!DOCTYPE html>
<html lang="hu">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>TypeScript gyakorlás</title>
</head>
<body>
    <h1>Első TypeScript alkalmazás</h1>
    <button id="btn">Kattints ide!</button>
    <p id="eredmeny"></p>
    <script src="main.js" defer></script>
</body>
</html>
```

A script elem a main.js fájlra hivatkozik. A böngésző a lefordított JavaScriptet futtatja. A main.js a fordítás után jön létre, ezért kézzel nem kell elkészíteni.

A defer attribútum hatására a külső JavaScript a HTML feldolgozásának befejezése után fut le, így a gomb és a bekezdés már elérhető a program számára.

A main.ts tartalma

```
const button = document.getElementById("btn") as HTMLButtonElement;
const eredmeny = document.getElementById("eredmeny") as
HTMLParagraphElement;

function koszont(nev: string): string {
    return `Szia ${nev}! Üdvözöllek a TypeScript világában!`;
}

button.addEventListener("click", () => {
    eredmeny.textContent = koszont("Hallgató");
});
```

A nev: string szöveges paramétert, a függvény utáni : string szöveges visszatérési értéket jelöl. Az as HTMLButtonElement és as HTMLParagraphElement típusállítás: a fordítónak megadjuk az elem típusát. Ez nem végez futásidejű ellenőrzést a HTML-azonosítóknak pontosan egyezniük kell.

Az addEventListener a gomb kattintásához rendel eseménykezelőt. A textContent a bekezdés szövegét módosítja. Mentsük mindkét fájlt a Ctrl + S billentyűkkel.

5 Fordítás JavaScriptre

A VS Code terminálját a projekt mappájában nyissuk meg. Az első fordításhoz futtassuk:

- tsc main.ts --target ES2017

A fordító létrehozza a main.js fájlt a main.ts mellett. A célverziót megadó --target ES2017 kapcsolóval a következő példához hasonló JavaScript készül:

```
function koszont(nev) {
    return `Szia ${nev}! Üdvözöllek a TypeScript világában!`;
}
```

A típusmegadások eltűnnek: a JavaScriptben nincs nev: string és visszatérési : string. A teljes main.js a DOM-elemek kiválasztását és a kattintás eseménykezelőjét is tartalmazza.

A rövidebb tsc main.ts parancs is fordít, de más alapértelmezett célverzió miatt a létrejövő kód alakja eltérhet. A fordítás után mindig nézzük meg a terminálban jelzett hibákat.

6 Az alkalmazás futtatása

Nyissuk meg az index.html fájlt böngészőben. Kattintsunk a Kattints ide! gombra. A gomb alatt ennek kell megjelennie:

```
Szia Hallgató! Üdvözöllek a TypeScript világában!
```

Ha megváltoztatjuk a main.ts tartalmát, mentsük el, fordítsuk újra, majd frissítsük a böngészőt. A böngésző mindig a JavaScript-fájlt tölti be, ezért a TypeScript módosítása önmagában nem elég.

7 Automatikus fordítás

A figyelő üzemmód minden mentés után újrafordítja a fájlt:

- tsc main.ts --target ES2017 --watch

A --watch rövidítése -w. A terminálban a sikeres fordítás után például a következő jelenik meg:

```
Found 0 errors. Watching for file changes.
```

Hagyjuk nyitva ezt a terminált. A fordító leállításához nyomjuk meg a Ctrl + C billentyűket. A figyelő üzemmód csak a fordítást végzi el, a böngészőoldalt önmagában nem frissíti.

8 Projektkonfiguráció létrehozása

A fordítási beállításokat a tsconfig.json fájlban tárolhatjuk. Állítsuk le a korábban indított figyelő módot, majd a projekt mappájában futtassuk:

- tsc --init

A létrejött tsconfig.json teljes tartalmát cseréljük az alábbira, majd mentsük el:

```
{
  "compilerOptions": {
    "target": "ES2017",
    "module": "none",
    "strict": true,
    "rootDir": ".",
    "outDir": "./js",
    "lib": ["ES2017", "DOM"],
    "noEmitOnError": true
  },
  "include": ["main.ts"]
}
```

Beállítás	**Jelentés**
target	A létrehozott JavaScript célverziója.
module	A none értékkel modul nélküli szkriptet készítünk.
strict	Szigorúbb típusellenőrzés.
rootDir	A forrásfájlok gyökérkönyvtára.
outDir	A generált JavaScript célmappája.
lib	Az ES2017 és a böngészős API-k típusdefiníciói.
noEmitOnError	Fordítási hiba esetén nem készül új JavaScript.
include	A projektbe bevont forrásfájlok.

Ezután a projekt fordítása és a figyelő mód indítása:

- tsc
- tsc -w

A fenti két parancs alternatíva: a tsc egyszer fordít, a tsc -w folyamatosan figyel. Konfiguráció használatakor ne adjunk meg forrásfájlnevet. A tsc main.ts parancs figyelmen kívül hagyja a tsconfig.json beállításait.

A JavaScript hivatkozásának módosítása

A fordítás eredménye most a js mappába kerül. Az index.html script elemét ezért cseréljük erre:

```
<script src="./js/main.js" defer></script>
```

A projekt a következő fájlokat tartalmazza:

Fájl	Szerep
index.html	A böngészőben megnyitott oldal.
main.ts	A szerkesztett TypeScript-forrás.
tsconfig.json	A projekt fordítási beállításai.
js/main.js	A fordító által létrehozott JavaScript.

Ha a korábbi lépésekből a projekt gyökerében is maradt main.js, azt törölhetjük. A HTML-nek már a js/main.js fájlt kell betöltenie. A generált JavaScript helyett mindig a main.ts fájlt szerkesszük.

9 Live Server használata

A Live Server helyi webszerveren jeleníti meg a HTML-oldalt és fájlváltozás után frissíti a böngészőt.

1. Nyissuk meg az Extensions panelt a Ctrl + Shift + X billentyűkkel.
2. Keressünk rá a Live Server bővítményre.
3. Telepítsük a Ritwick Dey által készített változatot.
4. Nyissuk meg az index.html fájlt.
5. Kattintsunk az alsó állapotsor Go Live gombjára. Alternatíva: jobb kattintás az index.html fájlon, majd Open with Live Server.

A böngészőben például a http://127.0.0.1:5500/index.html cím nyílik meg. A port száma eltérhet.

A napi munkafolyamat

Indítsuk el a terminálban a tsc -w parancsot, majd az oldalt a Live Serverrel. Módosítsuk és mentsük a main.ts fájlt. A fordító frissíti a js/main.js fájlt, a Live Server pedig frissíti az oldalt. Ha fordítási hiba van, először azt javítsuk ki: a böngészőben a korábbi JavaScript eredménye maradhat látható.

A folyamat tehát: TypeScript szerkesztése és mentése → JavaScript előállítása → oldalfrissítés → eredmény ellenőrzése a böngészőben.

10 Gyakori hibák és megoldásuk

A terminál nem találja a parancsot

Ha a tsc is not recognized üzenet jelenik meg, ellenőrizzük a TypeScript telepítését az npm install -g typescript paranccsal, majd indítsuk újra a VS Code-ot. Ha a node vagy az npm sem ismert, ellenőrizzük a Node.js telepítését és a PATH beállítást.

A PowerShell blokkolja az npm vagy a tsc indítását

Ha az npm.ps1 cannot be loaded vagy hasonló üzenet jelenik meg, a VS Code terminálpaneljén a + melletti nyíllal válasszuk a Command Prompt profilt. Másik megoldás a parancsok .cmd változata:

- npm.cmd install -g typescript
- tsc.cmd -v
- tsc.cmd -w

Nem jön létre vagy nem frissül a JavaScript

Mentsük a main.ts fájlt, nézzük meg a terminál hibáit és ellenőrizzük az outDir beállítást. A megadott konfiguráció a js mappába ír. A noEmitOnError miatt hiba esetén nem készül új kimenet.

Nem működik a gomb vagy nem látszik a változás

Ellenőrizzük a btn és eredmeny azonosítókat, a script fájlútvonalát, valamint azt, hogy lefutott-e a fordítás. A böngésző F12 billentyűvel megnyitható fejlesztői eszközeinek Console paneljén ellenőrizzük a hibákat. Szükség esetén frissítsünk Ctrl + F5 billentyűkkel.

A legfontosabb parancsok

Parancs	Feladat
node -v / npm -v	A telepítés ellenőrzése külön parancsokkal.
npm install -g typescript	A TypeScript globális telepítése.
tsc -v	A fordító verziójának ellenőrzése.
tsc main.ts	Egyszerű fordítás, alapértelmezett ES5 célverzióval
tsc main.ts --target ES2017	Egy forrásfájl fordítása ES2017 célverzióval.
tsc main.ts --target ES2017 -w	Egy forrásfájl figyelése.
tsc --init	Konfiguráció létrehozása.
tsc	Fordítás a projektkonfigurációval.
tsc -w	Automatikus fordítás a konfigurációval.

Ellenőrzés: a gomb működik, a terminál nem jelez fordítási hibát és a main.ts mentése után a böngészőben is megjelenik az új eredmény.